

ProRack software architecture

Market validation and decision report

Prepared for ProRack's business decision makers and software developer | 8 September 2026

Recommendation: continue with the TypeScript web architecture, with a smaller first release and several refinements. It fits the stated React/TypeScript skills, local-hosting preference and need to connect independent operational systems. The research does not establish a universally superior framework. The strongest competing approach is a Frappe custom app if the actual work is mainly inside ERPNext.

The proposed product is a browser-based operational portal and integration layer. Its value is connecting approved business facts across the digital thread, identifying exceptions and presenting evidence-backed project briefs. Buying or building another ERP, production scheduler or stock ledger would change this decision substantially.

Decision	Recommended direction
Keep	React + TypeScript; NestJS modular backend; PostgreSQL; Microsoft Entra sign-in; isolated integration worker; explicit source ownership.
Refine	Prefer React Router Framework Mode with static SPA deployment. Keep pg-boss provisional. Treat cloud hosting as an operating decision and AI routing as a measured policy.
Compare before a large build	A Frappe-native implementation of one representative ERP-heavy workflow against the same workflow in the proposed portal.
Validate first	Real ERP permissions, PowerFab API coverage and entitlement, project/revision mapping, data destinations and a named recovery owner.
Defer	Native Expo application, microservices, Temporal, a separate AI gateway and a high-volume machine data platform until requirements justify them.

What changed through research. Power Apps now supports code-first React applications; it is a credible cloud-managed alternative. BullMQ now documents a PostgreSQL backend, so Redis avoidance is no longer unique to pg-boss. React Router can add framework conventions without requiring a runtime SSR server. These findings refine the earlier blueprint rather than justify a wholesale rewrite.

Confidence: high in documented product capabilities; moderate in ProRack-specific fit. No live ERP, PowerFab, machine, identity or model benchmark was performed for this report. The framework recommendation is conditional on the operational and integration gates in this report.

Source notes: [Microsoft: Code Apps overview, updated 4 September 2026](#); [BullMQ: PostgreSQL backend](#); [React Router: SPA deployment](#). Undated documentation accessed 8 September 2026.

1. Which architecture wins under which conditions?

This comparison uses qualitative judgment, not invented numerical scores. Priority is given to source ownership, local deployment feasibility, available maintainers, permissions, recovery and total operating effort. Popularity, agent-generated-code volume and synthetic request-per-second benchmarks are weak proxies for those needs.

Approach	Where it can be better	Why it is not the unconditional choice
Custom React + NestJS	A distinct user experience joining ERP, fabrication, documents and site progress; one TypeScript application team.	ProRack owns authorization, integrations, monitoring, upgrades and recovery. A custom app is not inherently cheaper.
Frappe custom app	New ERP forms, linked records and approvals that can reuse existing users and business workflows.	Python/Frappe maintenance skills and ERP upgrade coupling; external source permissions and machine integrations still need design.
Power Apps Code Apps	Managed application hosting and Microsoft platform governance while retaining React/TypeScript development.	Cloud platform, Premium end-user licenses, connector constraints and industrial adapters remain.
React + ASP.NET Core	A committed C# team or partner, substantial supported .NET integrations and a preference for Microsoft's backend ecosystem.	A language change for the current developer; PowerFab's .NET API alone does not require every service to be .NET.
Appsmith / Retool	Fast administrative forms, tables and narrow internal applications with acceptable commercial terms.	Required SSO and self-hosted packages can dominate cost; complex permissions and specialized interactions remain application work.

Recommended decision rule: choose the custom TypeScript portal when its central feature is cross-system visibility. Choose Frappe-native when its central feature is extending ERP processes. Choose Power Apps Code Apps when reducing platform operations is a higher priority than local app hosting and its full user cost is acceptable. Choose .NET when the people maintaining the system make it the simpler long-term option.

Do not run all five as competing production platforms. A bounded comparison can use the same permitted project list and one installation checklist. Keep the winning workflow in one application, with one authoritative record and clear links to other systems.

Adjacent market options. Odoo is an ERP-selection alternative, not a drop-in frontend framework. Its commercial API access depends on plan; moving ERP would require a separate process-fit and migration study. Ignition is relevant to industrial data acquisition and visualization, not a substitute for the business portal. Neither should be purchased solely to solve the first project-list task.

Source notes: [Frappe: Apps](#); [Microsoft: ASP.NET Core overview](#); [Odoo: External JSON-2 API. version 19](#); [Inductive Automation: IIoT platform](#). Accessed 8 September 2026; comparative fit is the report's assessment.

2. Frontend and backend: retain the skills advantage

Frontend: React DOM + TypeScript, using React Router Framework Mode as a static SPA. This is a refinement of the Vite proposal: the framework itself uses a Vite plugin. Its route conventions, type support and code splitting reduce choices the developer otherwise has to assemble. React's own guidance generally favors a framework for new applications while explicitly allowing a build-from-scratch path.

SPA mode does not require a runtime server-rendering service. It still renders the root at build time, so browser-only APIs must be handled appropriately and private tenant data must never be baked into build output. Keep TanStack Query where it helps server-state caching, but define one owner for fetching and invalidation; route loaders and Query should not independently maintain competing copies.

Plain Vite with React Router Data Mode remains acceptable if that is easier for the maintainer. There is no existing app here requiring migration, and neither choice changes ERP integration feasibility.

Next.js is a valid alternative. It supports self-hosting with Node or Docker; it does not require Vercel. Adopt it if server-rendered customer/public pages, an experienced Next team or a coherent full-stack web design makes it useful. Do not add Next as another backend alongside Nest without assigning distinct responsibilities. The first internal project portal has no demonstrated requirement that makes it preferable.

Backend: keep NestJS as a modular monolith. Put project access, source adapters, evidence rules and workflow logic in clear modules. Run the worker separately from the API, sharing domain code. Nest provides application structure; it does not supply ProRack's permission rules or industrial domain model.

ASP.NET Core is the strongest whole-backend substitute when a C# owner is available. It has integrated authentication, authorization, dependency injection and operational facilities. Keep the initial PowerFab adapter narrow unless experience shows that maintaining separate Node and .NET processes costs more than consolidating. Confirm the vendor SDK's supported runtime independently.

FastAPI is useful if real Python processing dependencies dominate. Calling an AI service is not itself a reason to add a Python backend. Its in-process background task facility should not be treated as a durable integration queue.

For mobile work, retain Expo as a later option. Test actual tablet/browser/scanner and offline requirements first. React experience transfers, but web components, accessibility, CSS and large-table interaction still need deliberate work.

Source notes: [React team: Build from scratch](#); [React Router: Picking a Mode](#); [Next.js: Self-hosting](#); [NestJS documentation](#); [FastAPI: Background Tasks](#). Accessed 8 September 2026; no performance ranking was measured.

3. The serious alternatives: Frappe and Power Apps

Frappe deserves a real comparison, not automatic rejection. Custom apps can add controllers, hooks and migrations to an ERP site. Native roles, document permissions and field permission levels can reduce duplicated administration for ERP-owned workflows. Microsoft social login is documented, so Microsoft identity does not uniquely favor NestJS.

Frappe UI uses Vue 3. A React frontend can consume Frappe REST, but it does not automatically acquire the Frappe UI utilities. The decision is therefore broader than React versus Vue: weigh the cost of learning Frappe/Python against maintaining a new data model, session service and permission layer.

The best Frappe-native candidate is a missing installation or site-inspection workflow with ERP project links, assignments and approval history. The least persuasive reason to create a separate Nest application is merely to expose another editable copy of those records.

For the Company AI portal, a separate service still has a defensible purpose: combine permitted PowerFab, ERP and document evidence without putting every background job inside the ERP release and failure boundary. This is an architectural tradeoff, not a claim that Frappe cannot integrate other systems.

Power Apps Code Apps materially changes the Microsoft comparison. Microsoft's official repository describes general availability and supplies a React/Vite starter. Managed hosting and connector access may remove useful plumbing. However, current limitations include no Secure Implicit Connections and no Power Platform Git integration. Ordinary source control remains possible; these limitations do not mean Git cannot store the code.

The platform hosts the application. An on-premises data gateway connects local sources to cloud services; it does not make the application local. Keep secrets out of compiled frontend assets in every option. Source-level access through shared connections requires explicit design and verification.

Power Apps Premium is required for Code Apps end users. Existing M365 licensing should not be assumed to include custom or on-premises connector use. Dataverse is a separate data and permission system, not a transparent mirror of ERP permissions. Do not create new Dataverse stock or order masters simply because the platform makes tables easy to create.

Comparison gate: implement the same small workflow in Frappe and the proposed stack only if ERP ownership is genuinely undecided. Measure permission correctness, maintainable changes, deployment and recovery. Do not build two complete pilots or delay the basic ERP connectivity test.

Source notes: [Frappe: Users and Permissions](#); [Frappe UI introduction](#); [Frappe: Social Login Key](#); [Microsoft: Code Apps repository](#); [Microsoft: On-premises gateway](#); [Microsoft: Licensing FAQ](#). Accessed 8 September 2026.

4. Hosting, storage and identity are separate choices

Keep the local deployment baseline only with an operating owner. Use managed Linux infrastructure for the web/API/worker and PostgreSQL, plus a supported Windows service if the PowerFab integration requires it. Container packaging makes deployment repeatable; a single host still has a single-host outage boundary.

Document who patches the host, monitors jobs, renews certificates, restores databases and covers absence. Specify acceptable data loss and recovery time with the business. Test recovery to another machine. A workstation copy, NAS mirror or successful backup job does not establish that the application can be restored.

Supabase remains useful, but not mandatory. Self-hosted Supabase includes more services than PostgreSQL alone and transfers operations to the customer. Its production self-hosted stack differs from its local development environment; managed backups/PITR and several platform facilities are not included in the self-hosted offering. If only relational data and server-controlled access are needed, ordinary PostgreSQL is a smaller baseline.

Managed Supabase or Azure Database for PostgreSQL becomes a serious alternative when approved cloud hosting reduces an otherwise unsupported operating burden. Azure documents managed backups and configurable resilience, but the customer must select appropriate capabilities. That does not prove zero outages or automatically recover the application, files and edge adapters.

Keep application-owned files in private local storage initially. Access them through an authorization-checking backend and a small storage interface. Keep approved drawings in their existing authoritative repository where possible. Cloudflare R2 can be evaluated if cloud object storage is approved; an APAC location hint is not an India-only or plant-local guarantee.

Use Microsoft Entra for identity, then enforce business access in the application. The proposed backend-managed browser session remains appropriate. Verify tenant, user and roles; enforce project, field and document access on every relevant operation, including exports and AI retrieval. App roles are claims that the application must check.

Graph delegated access and scheduled application access have different permission boundaries. Selected permissions can restrict application access to approved resources, but do not by themselves prove that every portal user may read every retrieved document. Recheck permissions when serving cached evidence and generated briefs; revoke or expire sessions deliberately.

Local hosting still depends on the internet for fresh Entra authentication and approved cloud AI calls. Define degraded read-only behavior and visible freshness. Do not promise a fully offline system under this architecture.

Source notes: [Supabase: Self-hosting](#); [Microsoft: PostgreSQL reliability](#); [Cloudflare: R2 data location](#); [Microsoft: App roles](#); [Microsoft: Graph selected permissions](#). Accessed 8 September 2026.

5. Jobs, integration reliability and machine data

Keep a durable worker, but do not freeze the queue choice on outdated assumptions. pg-boss is a reasonable provisional choice because the proposed stack already contains PostgreSQL. It supports retries, scheduling and transactional job creation. Its README also discloses a single maintainer; that concentration is worth accounting for when assigning dependency ownership and upgrade testing.

BullMQ's current documentation includes a PostgreSQL backend. Its own guidance calls Redis the default and most battle-tested backend. Public documentation is not proof of maturity in the exact released version ProRack will install. Compare the supported release, retry behavior, queue visibility, migration process and recovery tests before changing the provisional choice. Do not deploy two queue engines for the pilot.

Temporal is a credible later choice for long-running workflows, external signals and compensation across multiple systems. It has a production service and operational requirements beyond its development server. Introduce it when durable business orchestration becomes a demonstrated need; a read-only project brief does not currently establish that need.

Neither a queue nor a workflow engine guarantees exactly-once external effects. For example, a source can accept a transaction while the caller loses the response. Proposed controls are stable operation identifiers, source acknowledgements, bounded retries and reconciliation. For the read-only pilot, these also prevent repeated model charges and duplicate evidence records where possible.

Record source ID, source revision, observed time, source-modified time, import status and mapping status separately. Preserve raw source meaning; do not silently reinterpret an ERP percentage as physical fabrication completion. Resolve cancellations, deletions and changed revisions instead of merely appending new snapshots.

Machine data is a different workload. Obtain each machine's protocol, register/tag definition, timestamps, units, reset behavior and sample payloads from the supplier. Keep local buffering and replay independent of the office dashboard. Distinguish cycle counts, accepted quantities, scrap and rework. Do not send every telemetry sample to the AI queue.

If multiple incompatible devices, historian needs or industrial visualization dominate, evaluate a supported industrial platform such as Ignition and its relevant modules against custom adapters. Its documented OPC UA/MQTT capabilities make it a credible shortlist candidate, not proof that a particular machine connects. Machine control and safety interlocks remain outside Company AI.

Source notes: [pg-boss: Maintainer README](#); [BullMQ: PostgreSQL backend](#); [Temporal: Production checklist](#); [Inductive Automation: OPC UA module](#). Accessed 8 September 2026; recovery controls are proposed design requirements, not completed tests.

6. AI: route by policy and evidence, then by cost

Your idea of using smaller models for simpler work is sound, but the first route should be **no model when a deterministic query or rule is sufficient**. Fetching current projects, computing totals and filtering overdue dates should use authorized APIs and application code.

Task class	Proposed execution	Required check
Project list, totals, status filters	ERP/API query plus deterministic rules	Correct source semantics and user scope.
Bounded extraction or short summary	Evaluated economical model	Schema validity and source-grounded fields.
Cross-source delivery-risk explanation	Evaluated stronger reasoning model	Evidence links, contradictions and explicit missing information.
Scanned documents or drawings	Separate OCR/vision workflow if justified	Revision, units and human verification of consequential fields.
Missing or stale evidence	Refresh, disclose gap or abstain	A stronger model must not invent the missing fact.

Use one approved provider and a small evaluated model set first. Keep the task policy in ProRack's backend: permitted user, permitted evidence, approved destination, cost/time budget and allowed tools. Only then call the provider. OpenAI's model-selection guidance favors meeting accuracy targets before optimizing cost and latency. This supports evaluation-driven routing, not choosing solely by token price.

Direct SDK versus gateway. Direct provider SDKs are sufficient for the initial application. The TypeScript AI SDK offers provider registries and aliases if multiple adapters become useful; use explicit approved provider configuration. LiteLLM is a reasonable later shared gateway for budgets, virtual keys and routing across several applications. Both still require ProRack's business authorization outside the model gateway.

OpenRouter is a hosted routing alternative, not something the user has already selected. Its provider controls include fallback, data-collection and zero-retention restrictions. Default routing settings should not be mistaken for company policy. Approve the router and every possible downstream destination, and constrain fallback before sending company content.

Azure Foundry deserves comparison if Microsoft procurement and access governance are useful. Validate the specific model, deployment type and processing geography. A resource's storage location does not establish inference location, and APAC processing is not synonymous with India-only processing.

Self-hosted models are an option for data that cannot leave approved infrastructure, but no evidence here establishes sufficient quality, GPU sizing or lower total cost. Keep that as a separate benchmark. API retention settings likewise need precise review: disabling response storage is not a blanket promise of zero retention.

Source notes: [OpenAI: Model selection](#); [AI SDK: Provider management](#); [LiteLLM: Gateway](#); [OpenRouter: Provider routing](#); [Microsoft: Foundry deployment types](#); [OpenAI: Data controls](#). Accessed 8 September 2026. AI SDK evidence came from indexed official documentation; its page body could not be opened by the research browser.

7. Test the choice against the digital thread

The table is a **proposed ownership model**, not a claim that all listed capabilities are installed or configured. Final ownership requires process-owner agreement. The portal should connect records and surface gaps; it should not create competing editable masters.

Stage	Proposed authoritative owner	What the custom layer must connect or prove
Design	Engineering CAD/Tekla and approved drawing repository	Project, assembly/part identity, released drawing and revision; avoid duplicate drawing masters.
BOM	Released engineering/fabrication BOM, mapped to ERP items	Revision approval, quantities, units and impact of change; define engineering versus purchasing views.
Procurement	ERP purchasing	Material request, PO and supplier commitments linked to released demand.
Raw material	Chosen receiving/stock authority, normally ERP; fabrication detail if assigned to PowerFab	Receipt, heat/batch/lot, certificate, usable quantity and agreed ownership boundary.
Production	PowerFab if its demonstrated workflow fits; otherwise explicitly chosen production system	Job, routing, planned versus actual progress and part-to-project links.
Machine	Machine/edge capture system	Timestamped events, good/scrap units, machine identity and job assignment.
QC	One selected quality workflow in ERP/production or a proven custom gap	Inspection against drawing revision, hold/release and nonconformance disposition.
Inventory	One authoritative stock ledger for each stock domain	Reconcile quantities and movements; avoid dual ERP/PowerFab posting ownership.
Shipment	ERP dispatch plus fabrication shipment details where needed	Package/assembly, delivery record and partial shipment traceability.
Installation	Existing suitable project tool or one custom/Frappe workflow	Site receipt, installed quantity, inspection, issue and signed evidence.
Project completion	Approved project closeout workflow	Outstanding defects, handover, completion acceptance and linked commercial status.

End-to-end acceptance scenario. Select one real project with a released design revision, material receipt, fabricated component and site delivery. Follow the IDs and quantities through every applicable stage. Change the design revision, record a QC rejection and split a shipment. The portal must show the resulting exceptions without treating them as unexplained quantity loss or silently overwriting history.

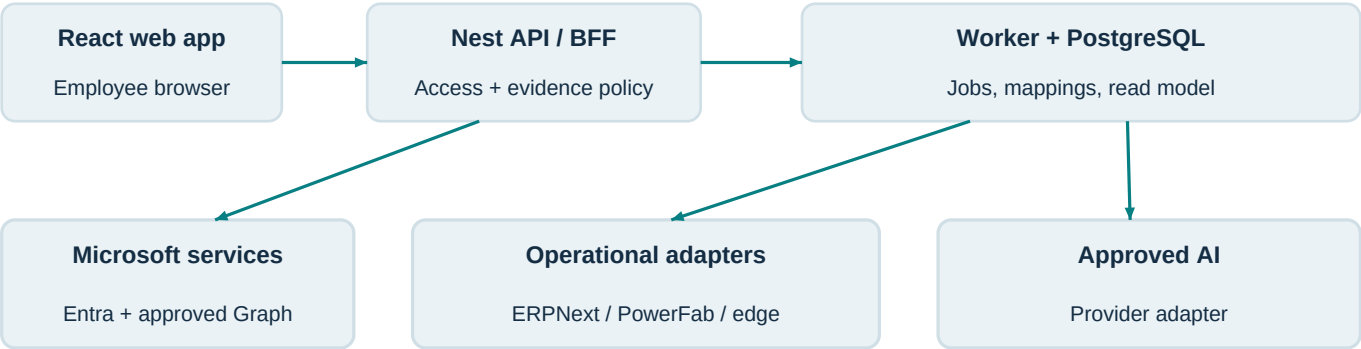
ERPNext documents serial/batch bundles linked to stock transactions, which is a useful native capability to investigate before custom-building traceability. Trimble documents supported APIs, but specifically distinguishes broad .NET access from limited REST. Those facts justify a mapping-and-capability proof, not an assumption that either product already covers the whole thread.

Source notes: [Frappe/ERPNext: Serial and Batch Bundle](#); [Trimble: Introduction to PowerFab APIs](#). Accessed 8 September 2026. Ownership and acceptance scenario are this report's proposals.

8. Revised software boundary and API map

Build one primary custom application, with a separate worker and narrowly scoped adapters. Keep the first release read-only. Add operational writes only after the owning system and acknowledgement/reconciliation behavior are agreed.

PROPOSED LOGICAL BOUNDARIES - READ-ONLY PILOT



Boundary	Interface and implementation direction
Browser to backend	Versioned HTTPS JSON API; backend session cookie; server-enforced access. SSE is optional for job progress.
Microsoft sign-in	Entra OpenID Connect authorization-code flow through a maintained identity library; tokens remain server-side in the BFF design.
ERPNext	Frappe REST: GET /api/resource/Project for discovery, with permissions and pagination. Confirm installed DocType fields and status semantics.
PowerFab	Use demonstrated REST endpoints or a supported .NET adapter over the documented local/VPN interface. Confirm SDK runtime, version and entitlement.
Approved M365 documents	Microsoft Graph with deliberate delegated or scheduled application permissions and document-level access handling.
Models	Backend provider adapter and task policy; keys never delivered to the browser.
Machines	Vendor-specific edge interface; normalized, deduplicated events sent to the worker. No universal protocol assumption.

Start with the project-list endpoint before creating a large synchronized database. Once multiple sources are necessary, PostgreSQL can hold source mappings, permitted projections, evidence metadata, jobs and audit records. Preserve the authoritative record link and freshness. It should not become a replacement stock ledger.

The existing ERP fetcher is a discovery aid. A successful mock test is not evidence that installed API fields, credentials or user permissions work. Ordinary Git should hold source, configuration templates and migrations; secrets, production exports and backups require separate handling. This research did not connect or push to GitHub.

Source notes: [Frappe: REST API](#); [Microsoft: OAuth authorization-code flow](#). PowerFab and Graph evidence is cited on preceding pages. Diagram shows the proposed logical architecture, not deployed infrastructure.

9. Economics: compare the required package

No credible total-cost winner can be calculated without user counts, support ownership, workload, vendor entitlements and local operating costs. The price of a database instance or model token is only one component.

Option	Verified commercial signal	Implication for this decision
Power Apps Code Apps	Code Apps requires Premium end users. Microsoft US page lists Premium at US\$20/user/month, paid yearly.	Count everyone who runs the app; existing M365 is not proof of entitlement. Confirm India/tenant pricing and additional usage.
Appsmith	Business US\$15/user/month; Enterprise US\$2,500/month for 100 users. SAML/OIDC is listed in Enterprise; air-gapped edition is an add-on.	Price the SSO requirement, not only the inexpensive tier. A free community edition also exists.
Retool self-hosted	Current official self-hosted offering is Enterprise and requires a license key.	Obtain a current quote rather than applying older low-tier self-hosted claims.
Custom TypeScript / Frappe	Hosting, maintenance, recovery, integrations and specialist support remain costs even without a new per-user application platform fee.	Do not label either option free to operate. Include dependency and ERP customization maintenance.

These are public price signals checked on 8 September 2026, not a ProRack quotation. Microsoft explicitly notes country/currency variation. Do not use the advertised 2,000-seat discount as a small-company assumption. Taxes, negotiated terms and additional capacity are not calculated here.

Use a three-year comparison: implementation and migration + subscriptions + infrastructure + backup/storage + support/maintenance time + model and integration usage + expected outage/recovery burden + exit costs. For existing licenses, distinguish incremental cost from already committed spend.

Segment people into developers, full operational users, occasional app users, report-only users, service identities and external installers. Then apply each vendor's actual terms. A read-only user is not automatically an unlicensed user. Power BI, PowerFab/Go and ERP entitlements remain separate questions; a portal does not automatically remove underlying access obligations.

For AI, measure cost per **accepted, useful brief**, including retries, OCR, retrieval and failed runs. A cheap model with frequent escalation can cost more than a capable model used once. No percentage saving is claimed without an evaluation on ProRack tasks.

The small-team economic risk is unbudgeted maintenance. A supported managed platform can beat custom infrastructure even with a subscription. Conversely, a large information-user population can make per-user platforms expensive. Obtain the actual counts before treating either outcome as established.

Source notes: [Microsoft: Power Apps pricing](#); [Appsmith: Pricing](#); [Retool: Self-hosted offering](#). Accessed 8 September 2026. Cost framework is proposed; no invented labor rates, FX conversion or ROI benchmark is used.

10. What to prove before committing to the full build

Use a short validation sprint after credentials, sample data and source owners are available. The following sequence is a recommendation, not a promised delivery date. Missing vendor access can dominate elapsed time regardless of framework.

Gate	Concrete evidence required	Decision if it fails
1. ERP access	Fetch every permitted current project, including pagination; compare against ERP UI for two different users and agreed status filters.	Fix API access/semantics before adding AI or another data store.
2. PowerFab coverage	Vendor demonstration of required job, part, progress and shipment fields; supported version/interface, entitlement and network path recorded.	Use a supported adapter/export or narrow scope; do not invent undocumented REST endpoints.
3. Permissions	An authorized user can read the sample project; another cannot obtain it by URL, search, export, cached brief or document link.	Stop wider rollout until denial works throughout the data path.
4. Traceability	One real project maps revisions, parts, material and shipment/site evidence; unmatched IDs appear as exceptions.	Resolve data ownership and mapping before dashboard expansion.
5. Recovery	Restart worker during a job; replay safely; simulate source outage; restore database and files onto a separate environment.	Assign operating ownership and repair recovery before operational dependence.
6. Model quality	Human-scored project briefs with correct evidence and deliberate missing/contradictory cases; record cost and latency.	Improve retrieval/rules or abstain; only change model after identifying the failure.

Model evaluation proposal: begin with a small curated set spanning ordinary projects, stale data, conflicting revisions, partial shipments, restricted documents and missing facts. Have engineering/operations agree the expected answer and unacceptable errors. Keep a held-out set for model changes. A small pilot is a selection tool, not proof of a population-wide accuracy rate.

Use deterministic checks for IDs, quantities, units and citation targets. Evaluate factual correctness, coverage, abstention, access leakage, usefulness, latency and cost. Start with a capable baseline; retain an economical route only when it meets the agreed task standard. Limit retries and escalation and log why they occurred.

Practical first deliverable: Microsoft sign-in, a permitted project list and a source-linked project page for two or three real projects. Next add a read-only delivery brief. Defer write-back, machine integrations and offline installation transactions to separately demonstrated needs.

Business owners should settle workflow ownership; the vendor should demonstrate interfaces; IT should own identity and recovery; engineering/site users should judge whether the output supports real decisions. Code generation alone does not replace those responsibilities.

Source note: [OpenAI: Model selection](#), accessed 8 September 2026, supports evaluating accuracy before optimization. All gates, scenarios and acceptance criteria on this page are proposed tests and have not been executed.

11. Final choice, switching triggers and research limits

Recommended starting stack: React Router Framework SPA + TypeScript; NestJS API and separate worker; PostgreSQL with Drizzle as the current data-access choice; provisional pg-boss; Microsoft Entra identity; private approved storage; verified ERPNext/PowerFab/Graph adapters; one approved model provider behind ProRack's policy. Keep existing UI library choices provisional until actual table, touch and accessibility needs are tested. This review did not establish a reason to replace the ORM or choose a Gantt vendor.

Change the recommendation when...	Prefer...
The majority of new workflows are ERP-owned and a Frappe maintainer is available.	Frappe custom app, avoiding a second implementation of those workflows.
Cloud use is approved and local platform operations cannot be supported.	A managed deployment of the custom app, or Power Apps Code Apps if connector/user economics fit.
A C# partner/team will own the system and .NET integration dominates.	ASP.NET Core as the main backend.
The product becomes a public/customer-facing experience with significant server-rendering needs.	Reassess Next.js or React Router server rendering.
Durable multi-day workflow coordination or industrial telemetry becomes the dominant workload.	Evaluate Temporal or an industrial platform for that specific boundary.
Strict data rules prohibit all external model processing.	Deterministic features first; separately benchmark a local inference deployment.

Research scope. This report evaluated current official documentation and commercial constraints for the most relevant architectural approaches. It used targeted discovery, two independent research lanes, consolidation of contradictions and direct spot-checks of consequential claims. It is not an exhaustive census of every product, a legal opinion on licenses, a security certification or a performance benchmark.

Sources were checked on 8 September 2026. Linked source notes identify publishers and available dates; undated product documentation reflects the version visible at access. Official sources are strong evidence for offered features and terms but their marketing claims are not neutral proof of implementation effort. No vendor ROI figures or forum anecdotes were used to estimate ProRack outcomes.

The starting materials were the earlier software blueprint, architecture analysis and supplied business discussion/proposal context. These are design inputs, not independent proof of deployed capabilities. Actual ERP/Fab versions, source ACLs, machine protocols, cloud approvals, staffing, user counts and quotations remain unverified. Some AI SDK pages were available only as indexed official text; repeated unavailable vLLM pages were not used as substantive evidence.

Research stopped when each shortlisted approach had enough evidence to establish its fit and constraints. The remaining decision-changing gaps require ProRack configuration, vendor demonstrations and working prototypes rather than more generic searches.